

Compiladores — Proyecto 2

Implementación de Lenguajes Extensibles mediante Adaptable Parsing Expression Grammars en C++

Ignacio Paolo Vidal Garcia — 202410281
Tamy Kristel Flores Guevara — 202310472
Federico Iribar Casanova — 202310321
Sergio Felipe Delgado Amado — 202310234

Julio 2026

Índice

1. Introducción	2
2. Fundamentos Teóricos	2
2.1. PEG	2
2.2. APEG	3
3. Análisis del paper	3
3.1. Modelo formal	3
3.2. Semántica de Parsing Expressions	3
3.3. Lenguaje de atributos	4
4. Implementación	5
4.1. Arquitectura del sistema	5
4.2. ParserState.h	5
4.3. Combinators.h	6
4.4. Type.h y Value.h	6
4.5. Apeg.h	6
4.6. MetaParser.h	7
4.7. Ejemplos de uso	8
5. Conclusiones y limitaciones	9
6. Referencias	10

1. Introducción

Este proyecto toma como base el artículo “*A Haskell Library for Adaptable Parsing Expression Grammars*” de Cardoso, Reis, Emiliano y Ribeiro (SBLP 2023) [1], el cual presenta una semántica monádica completa para las *Adaptable Parsing Expression Grammars* (APEG) y una implementación en Haskell capaz de reconocer el lenguaje de juguete extensible μ Sugar. El objetivo del presente trabajo es doble: primero, analizar en profundidad el modelo formal descrito en el paper (sintaxis abstracta, semántica de las expresiones de parsing y semántica del lenguaje de atributos que manipula gramáticas); y segundo, *portar* ese modelo desde Haskell hacia C++, reemplazando la mónada de estado (`State PureState`) por una implementación imperativa explícita basada en estructuras de datos inmutables por copia (*checkpoint/restore*) y punteros compartidos (`std::shared_ptr`).

El resultado es una biblioteca en C++ organizada en dos capas:

- Un **prototipo mínimo (MVP)** —`ParserState.h` y `Combinators.h`— que implementa un reconocedor booleano con *scoping dinámico* de reglas, suficiente para demostrar el mecanismo central de las APEG: registrar y usar reglas nuevas en tiempo de parsing, con *rollback* correcto ante backtracking.
- Un **motor completo** —`Apeg.h`, `Type.h`, `Value.h`— fiel al modelo del paper, en el que los valores son de primera clase (incluyendo gramáticas y lenguajes tipados), las expresiones de parsing sintetizan atributos, y existen los operadores de extensión de gramática ($\oplus$, G-ext) y de lenguaje ($\triangleright$, L-ext) con verificación de consistencia de tipos.

Adicionalmente, se implementó `MetaParser.h`, un compilador de sintaxis concreta que traduce texto (por ejemplo, `create loops { stmt -> 'repeat' . num . 'times' . '{' . stmt . '}' ; }`) a expresiones de parsing ejecutables. Esto ataca directamente una limitación que los propios autores reconocen en sus conclusiones: su prototipo Haskell no define una sintaxis concreta para escribir gramáticas APEG, sino un DSL embebido que resulta incómodo de usar.

2. Fundamentos Teóricos

2.1. PEG

Las *Parsing Expression Grammars* (PEG), introducidas por Ford [5], son un formalismo de reconocimiento (no generativo, a diferencia de las gramáticas libres de contexto) en el que cada regla determina de forma no ambigua cómo debe analizarse una entrada. Las operaciones básicas son:

- **Secuencia** ($p_1.p_2$): p_1 debe reconocer un prefijo y, a partir de lo que queda, p_2 debe reconocer el siguiente prefijo.
- **Elección ordenada** (p_1/p_2): se intenta primero p_1 ; solo si falla —y *sin* consumir entrada— se intenta p_2 . Esta prioridad estricta es la diferencia esencial frente a la ambigüedad potencial de una gramática libre de contexto.
- **Repetición** (p^*): aplica p tantas veces como sea posible (codicioso), sin fallar nunca.
- **Predicados de anticipación** ($\&p$, $!p$): verifican si p reconocería la entrada actual sin consumirla; permiten lookahead arbitrario.

Una propiedad crucial de las PEG es el *backtracking con descarte de efectos*: si una alternativa o una secuencia falla, cualquier cambio de estado que haya provocado debe deshacerse por completo antes de intentar la alternativa siguiente. Esta propiedad es la que las APEG deben preservar incluso cuando el “estado” incluye la propia gramática.

2.2. APEG

Las *Adaptable Parsing Expression Grammars* (APEG), formalizadas originalmente por Reis et al. [3, 4] y refinadas con un lenguaje de atributos por Cardoso et al. [2], extienden las PEG con la capacidad de **modificar la propia gramática durante el parsing**. Esto permite reconocer lenguajes que no son PEG-reconocibles con una gramática fija, como lenguajes en los que una construcción declara identificadores que solo son válidos *después* de su declaración (por ejemplo, “`def x; use x;`” debe aceptarse, pero “`use x; def x;`” debe rechazarse). Esto es exactamente lo que se demuestra en la Sección 4.7 con el ejemplo `apeg_demo.cpp`.

El mecanismo central es que el entorno de parsing Θ ya no contiene solo asignaciones de atributos, sino también la gramática activa. Cada regla puede:

- leer y escribir **atributos** (heredados/sintetizados),
- evaluar **restricciones semánticas** (*constraints*) que fallan sin consumir entrada,
- producir una **actualización** ($\vartheta \leftarrow e$) que computa un nuevo valor de gramática y lo instala como la gramática activa para el resto del parsing.

Como toda modificación de Θ debe descartarse ante un fallo (igual que el cursor de entrada), una gramática “crecida” dinámicamente dentro de una rama que termina fallando *no debe sobrevivir* al backtracking. Este es precisamente el comportamiento que se verifica en el `Adv Test 3` de `main.cpp`: una regla definida dentro de un `AndPredicate` (lookahead positivo) desaparece tras la restauración del checkpoint.

3. Análisis del paper

3.1. Modelo formal

La Figura 3 del paper [1] define la sintaxis abstracta de APEG. Una producción tiene la forma $r ::= \langle A \vartheta :: \tau^n \ \bar{e} :: \tau^m \rangle \rightarrow p$, es decir, un no terminal A con una lista de parámetros heredados tipados $\vartheta :: \tau^n$, una lista de expresiones de retorno (atributos sintetizados) $\bar{e} :: \tau^m$, y un cuerpo p . Las expresiones de parsing p incluyen: secuencia ($p.p$), elección (p/p), negación ($!p$), repetición (p^*), llamada a no terminal ($A \ \bar{e} \ \vartheta$), *bind* ($\vartheta = p$), *update* ($\vartheta \leftarrow e$), *constraint* ($?e$), terminal (a) y vacío (λ). El lenguaje de expresiones de atributos e (que manipula mapas, valores y —lo más importante— gramáticas) se separa en un nivel aparte, descrito en la Sección 3 del paper.

Nuestra implementación en `Apeg.h` traduce esta sintaxis abstracta a tipos de C++ de forma casi literal:

- p (expresión de parsing) $\rightarrow$ `using PExpr = std::function<PResult(State&)>`.
- $A \ \vartheta :: \tau^n \rightarrow p$ (regla con parámetros) $\rightarrow$ `using Rule = std::function<PResult(State&, const std::vector<Value>&>`.
- e (expresión de atributo) $\rightarrow$ `using AExpr = std::function<Value(State&)>`.
- El juicio $(\Theta, p) \rightsquigarrow_w z \triangleright \Theta'$ (Figura 4) se traduce a un valor de retorno `PResult{bool ok; Value val}`, junto con la mutación *in place* de `State& s`, cuyo `Checkpoint (cursor, grammar, env)` juega el papel de Θ .

3.2. Semántica de Parsing Expressions

Las Figuras 4 a 8 del paper dan la semántica operacional *small-step* de cada construcción. Los puntos que más influyeron en el diseño de nuestros combinadores fueron:

- **Empty/Term** (Figura 4): no modifican Θ . En C++, `Empty()` y `Lit(c)` no tocan `s.env` ni `s.grammar`, solo `s.cursor`.
- **Seq / $\neg$ Seq** (Figura 5): si p_1 o p_2 falla, la secuencia completa descarta *todos* los cambios sobre Θ , no solo el cursor. Esto es lo que motivó que `Sequence` (MVP) y `Seq` (Apeg.h) guarden un *checkpoint* *antes* de ejecutar la secuencia y lo restauren en cualquier punto de fallo, en vez de restaurar solo la posición de entrada.

- **Choice₁/Choice₂/¬Choice** (Figura 5): la segunda alternativa se ejecuta siempre sobre el Θ *original* (no sobre el resultante de la primera rama fallida). Nuestro **Choice** hace exactamente `s.restore(cp)` antes de probar la segunda rama.
- **Star/¬Star** (Figura 6): la repetición es codiciosa y descarta los cambios de entorno de la *última* aplicación fallida de p , no de todas las anteriores (las iteraciones exitosas persisten). Nuestra **Star/Many** hace **checkpoint** dentro del bucle, en cada iteración, y solo revierte la última.
- **Not** (Figura 6): es la *única* regla en la que, curiosamente, los cambios de entorno de una ejecución *fallida* de p sí se propagan (regla **Not**: el resultado usa Θ' , el entorno tras evaluar p , no el original Θ), mientras que en $\neg$ Not (predicado que tuvo éxito) el predicado completo falla. Nuestra implementación de **Not/NotPredicate**, en cambio, **siempre** restaura **Theta** al checkpoint original, sea cual sea el resultado interno de p — es una simplificación consciente y documentada en el código, discutida como limitación en la Sección 5.
- **Llamada a no terminal** (Figura 8): cada llamada evalúa las expresiones de argumentos $\bar{e}$ sobre el Θ del llamador, abre un entorno nuevo con $\bar{v}' \mapsto \bar{v}^n$ (los parámetros heredados) y el contexto de tipos Γ_{v_1} de la gramática obtenida como primer valor v_1 (el “atributo de lenguaje”), y al terminar restaura Θ salvo por los valores de retorno. Nuestro **call** implementa esto guardando `savedEnv = s.env` antes de invocar la regla y restaurándolo después, de modo que las variables locales del calleo no se filtran al llamador — el mismo *scoping* dinámico por pila que ya se prototipó en `ParserState::env` (un vector de mapas).

3.3. Lenguaje de atributos

La Sección 3 del paper (Figuras 9–13) define la semántica del lenguaje de atributos: literales, referencias, mapas (**A-map**, **U-map**, **L-map**), y —lo más relevante para APEG— los **operadores de extensión de gramática**:

- **G-ext** ($e \triangleleft e'$, Figura 10): compone dos valores de gramática $v_g \uplus v'_g$. Si ambas definen el mismo no terminal A , el resultado tiene $A \rightarrow p_1/p_2$ (elección entre ambas definiciones). No hay chequeo de tipos, es puramente sintáctico.
- **L-ext** (misma notación $\triangleleft$, sobrecargada sobre valores de *lenguaje*): además de componer, re-chequea la consistencia de tipos de las firmas de reglas compartidas en el contexto de tipos Γ , produciendo Γ' . Solo si este chequeo pasa el resultado es un **tLang** válido.

Estos dos operadores se implementaron directamente en `Apeg.h` como `composeGrammars` (G-ext) y `extendLanguage` (L-ext), esta última lanzando una excepción `TypeError` cuando dos definiciones del mismo no terminal declaran aridades o tipos incompatibles —ver Sección 4.5.

Respecto a la **implementación monádica en Haskell** descrita en la Sección 4 del paper (`PureState`, `APegSt = State PureState`, y combinadores como `kleene`, `sequential`, `alternate`, `notPeg`, `constraintApeg`, `bindApeg`, `update`), el rol de la mónada de estado es exactamente el mismo que cumple nuestro `State&` pasado por referencia junto con `save()/restore()`: encapsular “obtener estado, aplicar acción, eventualmente reponer estado con put”. La correspondencia directa es:

Haskell (paper, Fig. 15–18)	C++ (este proyecto)
<code>get / put</code>	<code>state.save() / state.restore(cp)</code>
<code>onSuccess suc fai</code>	rama <code>if (r.ok)...</code> <code>else ...</code> tras la llamada
<code>kleene</code>	Star / Many
<code>sequential</code>	Seq / Sequence
<code>alternate</code>	Choice
<code>notPeg</code>	Not / NotPredicate
<code>constraintApeg</code>	Constraint
<code>bindApeg</code>	Capture
<code>update</code>	Bind / Update

La diferencia esencial es que Haskell obtiene composicionalidad y pureza “gratis” vía la mónada (do-notation desazucara a >>=), mientras que en C++ esa composicionalidad se logra explícitamente: cada combinador es una función de orden superior que *captura por valor* sub-expresiones (`std::function`) y maneja manualmente el guardado/restauración del estado. No existe mutabilidad compartida peligrosa porque `Grammar` se trata como inmutable por convención (`Grammar::with` retorna una copia) y se referencia mediante `std::shared_ptr<Grammar>`, que es nuestro análogo de la persistencia estructural que Haskell obtiene de forma nativa.

4. Implementación

4.1. Arquitectura del sistema

El sistema se organiza en dos niveles independientes, más una capa de sintaxis concreta:

1. **Nivel MVP** (`ParserState.h + Combinators.h`): un reconocedor booleano (`Rule = std::function<bool(ParserState&>>`) con *scoping dinámico* de reglas mediante una pila de mapas (`env`). Se usa en `main.cpp` para validar, de forma aislada y sencilla de razonar, las propiedades de backtracking sobre el entorno de reglas (la parte más delicada y menos trivial de APEG).
2. **Nivel completo** (`Apeg.h + Type.h + Value.h`): fiel al modelo del paper. Valores de primera clase (`Value`), gramáticas tipadas como valores (`Grammar, LanguageValue`), atributos heredados y sintetizados, operadores $\exists/\triangleright$ con chequeo de tipos. Se usa en `apeg_demo.cpp` para construir lenguajes extensibles reales.
3. **Sintaxis concreta** (`MetaParser.h`): un compilador recursivo-descendente que traduce texto con la gramática de patrones (secuencia `.`, elección `/`, repetición `*`, negación `!`, literales entre comillas, llamadas por nombre de identificador) a un `PExpr` ejecutable sobre el motor completo, y una función `parseCreate` que compila bloques `create <nombre> { regla -> patrón ; ... }`, análogos al `define` de μ Sugar (Figura 1–2 del paper) pero como sintaxis concreta real, en vez de un DSL embebido en el lenguaje huésped.

4.2. ParserState.h

Este archivo define el estado del prototipo MVP. Su pieza más interesante es el entorno de reglas como *pila de scopes*:

```

1 struct ParserState {
2     std::string_view input;
3     size_t cursor = 0;
4     std::vector<std::unordered_map<std::string, Rule>> env;
5
6     void push_scope() { env.emplace_back(); }
7     void pop_scope() { if (env.size() > 1) env.pop_back(); }
8
9     void register_rule(const std::string& name, Rule rule) {
10         env.back()[name] = std::move(rule);
11     }
12
13     bool execute_rule(const std::string& name) {
14         for (auto it = env.rbegin(); it != env.rend(); ++it) {
15             auto found = it->find(name);
16             if (found != it->end()) return found->second(*this);
17         }
18         return false;
19     }
20 };

```

Listing 1: Entorno de reglas con scoping dinámico (ParserState.h)

La búsqueda de `execute_rule` recorre la pila desde el scope más interno (`rbegin`) hasta el global (`rend`), lo que implementa *shadowing* correcto: una regla registrada en un scope local oculta temporalmente a una de igual nombre en un scope externo —esto se verifica en `Test 2` de `main.cpp`. El `Checkpoint` guarda una copia *completa* del vector de mapas (`env`), de modo que `restore` deshace tanto el cursor como cualquier regla registrada

dinámicamente durante la rama que falló — esta es la traducción directa, a nivel de prototipo, de la regla $\neg\text{Seq}/\neg\text{Choice}$ del paper (descartar cambios de Θ).

4.3. Combinators.h

Los cinco combinadores del MVP —MatchChar, Sequence, Choice, Many, AndPredicate/ NotPredicate— son la contraparte imperativa de las reglas de las Figuras 4 a 6. Por ejemplo, Sequence implementa **Seq** y $\neg\text{Seq}_{1,2}$ en un solo bloque:

```

1 inline Rule Sequence(Rule a, Rule b) {
2     return [a = std::move(a), b = std::move(b)](ParserState& state) -> bool {
3         auto checkpoint = state.save();
4         if (a(state)) {
5             if (b(state)) return true;
6         }
7         state.restore(checkpoint); // deshace cursor y entorno de reglas
8         return false;
9     };
10 }

```

Listing 2: Sequence: éxito compuesto o rollback total (Combinators.h)

Many (Kleene star) incluye *protección contra bucles infinitos*: si una iteración exitosa no avanza el cursor, se detiene —una salvaguarda pragmática que el modelo formal no necesita (porque asume gramáticas “bien formadas”), pero que es indispensable en una implementación real, dado que el propio paper reconoce en sus conclusiones que no implementa el chequeo de buena formación de Cardoso et al. AndPredicate y NotPredicate son ambas *no consuntivas*: guardan el checkpoint, ejecutan la subregla, y **siempre** restauran, retornando el resultado (o su negación).

4.4. Type.h y Value.h

Type modela el contexto de tipos $\Gamma : \varnothing \rightarrow \tau$ del paper con un `enum class TypeKind` que incluye, además de los tipos base (Int, Bool, Str, Sym, List, Node), los dos tipos que hacen a APEG distintiva: Grammar (τ_{Grm}) y Lang (τ_{Lang}) — el hecho de que una gramática y un lenguaje tipado sean, ellos mismos, tipos de primera clase es justamente lo que permite pasar gramáticas como argumentos y devolverlas como atributos sintetizados.

Value es el dominio semántico $\mathcal{V}$ de valores en tiempo de ejecución: una unión etiquetada (`enum class Kind`) con payloads para enteros, booleanos, cadenas, símbolos, listas, nodos de AST, y —de nuevo— `std::shared_ptr` `<Grammar>` / `std::shared_ptr<LanguageValue>` para que una gramática pueda viajar como un valor cualquiera (por ejemplo, ser almacenada en un atributo, pasada como parámetro heredado a un no terminal, o devuelta como resultado sintetizado). Los métodos `toString()` y `toTree()` se usan en las demos para imprimir el AST resultante de un parse exitoso.

4.5. Apeg.h

Es el motor completo. Sus piezas principales, y su correspondencia con el modelo formal:

```

1 struct PResult { bool ok; Value val; }; // juicio (~>_w z |> Theta')
2 using Rule = std::function<PResult(State&, const std::vector<Value>&)>;
3 using PExpr = std::function<PResult(State&)>;
4 using AExpr = std::function<Value(State&)>;
5
6 struct Grammar {
7     std::map<std::string, Rule> rules; // A -> p
8     std::map<std::string, Value> attrs; // atributos a nivel de gramática
9     std::map<std::string, RuleSig> sigs; // firmas para L-ext
10    Grammar with(const std::string& name, Rule rule) const; // copia inmutable
11 };
12
13 struct State {
14     std::string_view input;
15     size_t cursor = 0;
16     std::shared_ptr<Grammar> grammar; // gramática "adaptable"

```

```

17     std::map<std::string, Value> env; // atributos locales
18     struct Checkpoint { size_t cursor; std::shared_ptr<Grammar> grammar;
19                         std::map<std::string, Value> env; };
20     Checkpoint save() const;
21     void restore(const Checkpoint&);
22 };

```

Listing 3: Tipos centrales del motor APEG (Apeg.h)

El punto clave de diseño es que `State::grammar` es un `shared_ptr<Grammar>`: actualizar la gramática (operador `Update`) no muta el `Grammar` existente, sino que construye uno *nuevo* vía `Grammar::with(...)` y reasigna el puntero. Esto reproduce, en C++, la inmutabilidad estructural que en Haskell viene dada de forma nativa: un `Checkpoint` tomado *antes* de la actualización sigue apuntando al `Grammar` antiguo, así que un `restore` posterior deshace la extensión de la gramática automáticamente —sin necesidad de un mecanismo de deshacer explícito, igual que en el paper la regla `¬Choice` usa el Θ original.

Los operadores de composición se implementan así:

```

1  inline Grammar composeGrammars(const Grammar& g1, const Grammar& g2) {
2      Grammar result = g1;
3      for (const auto& [name, rule] : g2.rules) {
4          auto it = result.rules.find(name);
5          if (it == result.rules.end()) result.rules[name] = rule;
6          else it->second = ruleChoice(it->second, rule); // A -> p1 / p2
7      }
8      return result;
9  }
10
11 inline std::pair<Grammar, Gamma> extendLanguage(
12     const Grammar& g1, const Gamma& gamma1, const Grammar& g2) {
13     Grammar merged = composeGrammars(g1, g2);
14     for (const auto& [name, sig2] : g2.sigs) {
15         auto it = g1.sigs.find(name);
16         if (it != g1.sigs.end())
17             apeg_detail::checkSigCompatible(name, it->second, sig2); // lanza TypeError
18     }
19     /* ... extiende Gamma con los tipos de g2 ... */
20     return {std::move(merged), std::move(gammaPrime)};
21 }

```

Listing 4: G-ext y L-ext (Apeg.h)

`composeGrammars` es exactamente la regla **G-ext**: cuando dos gramáticas definen el mismo no terminal, el resultado es la elección ordenada de ambas definiciones (`ruleChoice`). `extendLanguage` añade encima el chequeo de consistencia de tipos que distingue **L-ext** de **G-ext**: si dos definiciones del mismo no terminal declaran aridades o tipos incompatibles en sus parámetros o retornos, se lanza `TypeError` — una verificación estática mínima pero fiel al espíritu de la Figura 10 del paper (evitar que se promueva a `tLang` una gramática cuyas firmas ya no son consistentes entre sí).

4.6. MetaParser.h

Mientras el paper implementa μ Sugar construyendo el AST directamente con un DSL embebido en Haskell (funciones como `rule`, `call`, `.=.`, `.<.` de la Figura 19–20), nuestro `MetaParser.h` da un paso más: define una **sintaxis concreta real** para escribir patrones PEG como texto, que se compila en tiempo de ejecución a un `PEExpr`:

```

1 pattern := seq ('/' seq)* // eleccion ordenada
2 seq     := prefix ('.' prefix)* // secuencia
3 prefix  := '!' term | term // negacion (lookahead)
4 term    := factor '*' | factor // estrella de Kleene
5 factor  := '\'' literal '\'' | '(' pattern ')' | identifier

```

Listing 5: Gramática de patrones soportada por MetaParser.h

El compilador es descendente-recursive clásico (`compilePattern` $\rightarrow$ `compileSeq` $\rightarrow$ `compilePrefix` $\rightarrow$ `compileTerm` $\rightarrow$ `compileFactor`), con backtracking manual sobre la posición del Scanner (`sc.pos`) idéntico en espíritu al que se usa para parsear el propio lenguaje de entrada. La función `parseCreate` compila bloques

```
1 create <nombre> {
2     <no_terminal> -> <patron> ;
3     ...
4 }
```

en un Grammar completo — el equivalente concreto del `define` de μ Sugar (Figura 1, regla `newSyn`). A diferencia del paper, que en su Sección 4 señala explícitamente como limitación futura no tener “una interfaz más amigable para escribir gramáticas APEG”, nuestra implementación sí resuelve ese punto para el subconjunto de PEG puro (secuencia, elección, estrella, negación, literales, llamadas); lo que *no* cubre todavía es una sintaxis concreta para los operadores de atributos (*bind*, *update*, *constraint*), que en nuestro motor solo son accesibles construyendo un `PExpr` en C++ directamente (ver `apeg_demo.cpp`).

4.7. Ejemplos de uso

`main.cpp` contiene seis pruebas sobre el prototipo MVP que verifican, una por una, las propiedades semánticas discutidas en la Sección 3:

- **Test 1** — secuencia simple (**Seq**).
- **Test 2** — *scoping* dinámico: una regla `var` registrada en un scope local oculta a la del scope global mientras el scope está activo, y la global reaparece intacta tras `pop_scope()`.
- **Test 3** — *backtracking semántico*: dentro de un `Choice`, la primera rama define dinámicamente la regla `var` y luego falla; la segunda rama vuelve a definir `var` (con el mismo valor) y tiene éxito. Se verifica que la definición que *persiste* es la de la rama ganadora, no un residuo de la rama fallida — la regla **Choice**₁ del paper.
- **Adv Test 1** — Kleene star con protección de bucle infinito sobre una regla que no consume entrada.
- **Adv Test 2** — predicados `AndPredicate` / `NotPredicate` como lookahead no consuntivo.
- **Adv Test 3** — el caso más sutil: `AndPredicate(DefineVar)` sobre “+x” tiene éxito (el lookahead positivo confirma que `DefineVar` reconocería la entrada), pero al ser no consuntivo **debe** descartar la regla `var` que `DefineVar` registró internamente. El test confirma que, tras el predicado, `UseVar` vuelve a fallar — validando exactamente el comportamiento de rollback total de Θ ante un predicado, incluso cuando el predicado internamente tuvo éxito.

`apeg_demo.cpp`, sobre el motor completo, construye un lenguaje de “declarar y luego usar” (**DemoA**) imposible de reconocer con una PEG estática:

```
1 Constraint([](State& s) -> Value {
2     std::string id = s.env.at("id").s;
3     auto it = s.grammar->attrs.find("declared");
4     if (it != s.grammar->attrs.end())
5         for (const auto& d : it->second.items)
6             if (d.s == id) return Value::Bool(false);    // ya declarado -> falla
7     return Value::Bool(true);
8 }),
9 Update([](State& s) {
10     std::string id = s.env.at("id").s;
11     Rule oldDeclared = s.grammar->rules.at("Declared");
12     Rule newDeclared = [id, oldDeclared](State& st, const std::vector<Value>&) {
13         auto cp = st.save();
14         PResult r = Word(id)(st);
15         if (r.ok) return r;
16         st.restore(cp);
17         return oldDeclared(st, {});    // encadena con las declaraciones previas
18     };
19     Grammar g2 = s.grammar->with("Declared", newDeclared);
```



```

20  /* ... registra "id" en g2.attrs["declared"] ... */
21  s.grammar = std::make_shared<Grammar>(std::move(g2));
22  })

```

Listing 6: Extracto: la gramática crece con cada declaración (apeg_demo.cpp)

Aquí `Constraint` rechaza redefiniciones (usando un atributo de *gramática*, que persiste entre sentencias porque vive en `Grammar::attrs` en lugar de en el `State::env` local de la llamada), y `update` *encadena* la regla `Declared` anterior con el nuevo identificador reconocido —de modo que `"def x; def y; use x; use y;"` se acepta pero `"use z;"` sin declaración previa se rechaza. Este es, en miniatura, el mismo mecanismo con el que μ Sugar activa la sintaxis `sfor` solo dentro de un bloque `syntax sfor { ... }` (Figura 2 del paper): una regla que antes no existía (o no aceptaba nada) se vuelve disponible a partir de cierto punto del parsing, y dicha disponibilidad respeta el backtracking.

5. Conclusiones y limitaciones

Este proyecto logró portar el modelo formal de APEG del paper de Cardoso et al. [1] —incluyendo la semántica de secuencia, elección, repetición, predicados, llamadas a no terminal con paso de atributos heredados, y los operadores de composición/extensión de gramáticas $\uplus/\triangleright$ con verificación mínima de tipos— desde una mónada de estado en Haskell hacia una implementación imperativa explícita en C++ basada en *checkpoints* inmutables y punteros compartidos. Se verificaron experimentalmente, mediante los tests de `main.cpp`, las propiedades más delicadas del modelo: el *scoping* dinámico de reglas, la persistencia de una gramática extendida únicamente en la rama ganadora de un `choice`, y el rollback total de la gramática (incluyendo reglas registradas internamente) tras un predicado de anticipación. Además, se construyó un ejemplo de lenguaje extensible (`apeg_demo.cpp`) que demuestra un caso de parsing sensible al contexto —*declarar-antes-de-usar*— irreconocible con una PEG estática, y se implementó `MetaParser.h`, un compilador de sintaxis concreta que resuelve directamente una limitación que los propios autores del paper dejan pendiente para trabajo futuro.

Las limitaciones conocidas de nuestra implementación son:

- **Sin chequeo de buena formación (*well-formedness*).** Igual que el prototipo Haskell del paper, no implementamos el análisis estático de Cardoso et al. [2] que garantiza terminación; solo mitigamos el caso más común (repetición sobre una regla que no consume entrada) con protección de bucle en tiempo de ejecución (*Star/Many*). Una recursión indirecta entre no terminales sin consumo de entrada podría seguir causando desbordamiento de pila.
- **Semántica simplificada de `Not`.** A diferencia de la regla `Not` del paper (Figura 6), que propaga el entorno Θ' resultante de una evaluación *fallida* de p , nuestro `Not/NotPredicate` siempre restaura el checkpoint original sin importar el resultado interno. En la práctica esto rara vez importa porque un p que fracasa típicamente no altera Θ de forma significativa, pero es una divergencia formal documentada.
- **Retorno de atributos sintetizados simplificado en `call`.** El paper especifica que, tras una llamada a no terminal, Θ' incluye los mapeos de Θ salvo los de $\bar{\vartheta}$, que pasan a sus valores de retorno correspondientes (Figura 8). Nuestra implementación retorna el valor sintetizado como `PResult::val` en lugar de escribirlo de vuelta en variables con nombre del entorno del llamador; quien llama debe capturarlo explícitamente con `Capture`.
- **Sistema de tipos mínimo.** `RuleSig` solo compara aridad y tipos declarados posición a posición; no hay inferencia de tipos ni polimorfismo, a diferencia de lo que insinúa el contexto Γ del paper en su forma más general.
- **`MetaParser.h` cubre solo PEG puro.** La sintaxis concreta no expone todavía *bind*, *update* ni *constraint* como texto, por lo que un lenguaje extensible con reglas semánticas (como el `DemoA`) aún debe escribirse directamente en C++ sobre `Apeg.h`; extender la gramática de `MetaParser.h` para cubrir el resto de μ Sugar (Figura 1) queda como trabajo futuro natural.

6. Referencias

Referencias

- [1] Elton M. Cardoso, Leonardo V. S. Reis, Marcos Geraldo Braga Emiliano, and Rodrigo G. Ribeiro. *A Haskell Library for Adaptable Parsing Expression Grammars*. In XXVII Brazilian Symposium on Programming Languages (SBLP 2023), Campo Grande, MS, Brazil, 2023. <https://doi.org/10.1145/3624309.3624313>
- [2] Elton M. Cardoso, Rodrigo G. Ribeiro, Leonardo V. S. Reis, Mariza A. S. Bigonha, Roberto S. Bigonha, and Vladimir O. Di Iorio. *An Attribute Language Definition for Adaptable Parsing Expression Grammars*. In Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing (SAC'19), 2019, pp. 1518–1525. <https://doi.org/10.1145/3297280.3299738>
- [3] Leonardo V.S. Reis, Roberto S. Bigonha, Vladimir O. Di Iorio, and Luis Eduardo S. Amorim. *Adaptable Parsing Expression Grammars*. In Programming Languages, LNCS vol. 7554, Springer, 2012, pp. 72–86.
- [4] Leonardo V.S. Reis, Roberto S. Bigonha, Vladimir O. Di Iorio, and Luis Eduardo S. Amorim. *The formalization and implementation of Adaptable Parsing Expression Grammars*. Science of Computer Programming, 96(2), 2014, pp. 191–210.
- [5] Bryan Ford. *Parsing Expression Grammars: A Recognition-based Syntactic Foundation*. In Proceedings of the 31st Symposium on Principles of Programming Languages (POPL'04), 2004, pp. 111–122.